ELI — Capability Catalogue

ELI v2.5.27

October 2026

Contents

ELI Capability Catalogue — every action & module, what it actually does **2**
Headline finding: 230 is real but aliased . . . 3
1. Conversation & reasoning . . . 3
2. OS & application control . . . 3
3. Input & screen control . . . 4
4. Gaze (webcam eye-tracking) . . . 4
5. Media . . . 4
6. Files & documents . . . 5
7. Vision & screen understanding . . . 5
8. Coding & code-repair . . . 5
9. Self-management & self-improvement . . . 5
10. Introspection & audits (grounded, deterministic) . . . 6
11. Memory & identity . . . 6
12. Habits, proactivity & goals . . . 6
13. Self-healing remediation (Linux) . . . 7
14. Voice & transcription . . . 7
15. Time, timers & utility . . . 7
16. Plugins (management) . . . 7
17. Plugin-backed capabilities . . . 8
Reading note . . . 8

Part 2 — `runtime/` module catalogue (94 files, 36.4k lines) **8**
Grounding & anti-confabulation . . . 9
Self-honesty / introspection / self-reporting . . . 9
Self-improvement & code-awareness . . . 10
Self-healing remediation . . . 10
Awareness & boot . . . 10
Autonomy / operator (governed) . . . 11
Response surfaces & governance . . . 11
Personal-memory surfaces . . . 12
Typed pipeline plumbing (evidence/packets) . . . 12
Security . . . 13

Part 3 — `cognition/` module catalogue (47 files, 20.8k lines) **13**
Orchestration & agents . . . 13
Inference . . . 14
Reasoning & engagement . . . 14
Context & grounding . . . 14
Persona (the living voice) . . . 15
Output governance (consolidated this session) . . . 15

Profile . . . 15

Part 4 — remaining packages (module catalogue) 15
kernel/ (the engine + boot) — 17.5k lines . . . 15
core/ (paths, settings, hardware, safety) — 11.2k lines . . . 16
memory/ — 8.8k lines . . . 17
perception/ — 10.2k lines . . . 18
planning/ (autonomy, habits, proactive) — 4.3k lines . . . 18
coding/ (the frontier coder) — 2.1k lines . . . 19
learning/ (real LoRA self-training) — 4.3k lines . . . 19
contracts/, integrations/, system/, utils/, cli/ . . . 20
world/ (embodied self-model) — 1.8k lines . . . 20
gui/ (PySide6 desktop) — 27.4k lines . . . 21
plugins/ — 10 built-ins + manager . . . 21

Part 7 — modules not covered above 22
runtime/ . . . 22
cognition/ . . . 25
execution/ . . . 26
core/ . . . 28
memory/ . . . 29
perception/ . . . 29
planning/ . . . 29
coding/ . . . 30
learning/ . . . 30
integrations/ . . . 30
system/ . . . 31
utils/ . . . 31
onboarding/ . . . 31
setup/ . . . 31
world/ . . . 32
gui/ . . . 33
plugins/ . . . 33
tools/ . . . 35
package root . . . 36
scripts/ . . . 36

Part 5 — engine request lifecycle (engine.process) 36

Part 6 — reasoning modes & autonomous deepening 36

Updated 2026-09-30. Gradient orchestrator for all CHAT modes; shared memory/retrieval.py; canonical S01–S12 via pipeline_trace.py. RESUME_TASK and MULTI_COMMAND's dependency/crash-survival layer added; agent-bus confidence and habit learning gained relevance/source integrity checks — see runtime_planning_world.md, orchestration_and_agents.md, dag.md.

ELI Capability Catalogue — every action & module, what it actually does

Purpose. A systematic, ground-truth catalogue built by reading the real handlers and modules — not summarised from memory. It exists because conversational summaries of a 195,242-line project (eli/, measured 2026-09-30) keep undershooting; this is the persisted, exhaustive map.

Method. Action list comes from the live `capability_manifest.json` (**230** entries; 189 routable, 207 in the executor's supported list, 213 in either), verified against the `executor_enhanced.py` dispatch. **The always-current, auto-generated action list with activation phrases is `capabilities_and_actions.md`** — this catalogue is the deeper module-level read. Behaviour grounded in the handlers and the subsystem blueprints. Where a description is inferred from name+structure rather than a line-by-line handler read, it is marked *(inferred)*.

Coverage: every action in the manifest (Part 1), every module under `eli/` (Parts 2–4 and 7).

Headline finding: 230 is real but aliased

The manifest's 230 entries are honest (*measured* by `capability_sync`, not asserted) but inflated by **alias families** — multiple action names routing to one behaviour. Collapsed, there are roughly **~110 distinct capabilities**. Alias families are grouped below so the real surface is visible.

---

1. Conversation & reasoning

Action(s)	What it does
`CHAT`	The default — full cognitive pipeline (router → gradient orchestrator → mode algo → governed output).
`ANSWER`, `DIRECT_RESPONSE`, `SAY`	Direct/short response surfaces (lighter than CHAT). *(inferred for some)*
`SET_AI_MODE`	Set the reasoning mode (Quick / Normal / Advanced / Research / Expert). Legacy names still accepted as input.
`SEQUENCE`	Multi-step action chaining (run a sequence of actions).
`MULTI_COMMAND`	The real chained-command executor ("close steam and set an alarm"). Router's `command_splitter.py` splits; the executor runs in topological layers by inferred dependency (`command_dependency_graph.py`, `eli/core/dag`), durably records each step's outcome (`command_sequence_log.py`) so a crash mid-sequence isn't a silent loss and a resubmit within an hour skips steps already known to have succeeded. `SEQUENCE` above is a different, simpler mechanism.
`NOOP`	No-op.

2. OS & application control

Action(s)	What it does
`OPEN_APP` = `OPEN_APPLICATION` = `LAUNCH_APP`	Launch an app, resolved via the `system_index` (your machine's executables — thousands, indexed live) + `portable_app_control`. On failure → **remediation offer** (real apt/snap/flatpak install on confirmation).

Action(s)	What it does
`CLOSE_APP` = `CLOSE_APPLICATION` = `EXIT_APP` = `QUIT_APP`	Close/quit an app.
`FOCUS_APP`, `HIDE_APP`	Focus / hide a window.
`MINIMISE_APP` = `MINIMIZE_APP`, `MINIMISE_WINDOW` = `MINIMIZE_WINDOW`, `MAXIMISE_WINDOW`	Window state control (UK/US spellings aliased).
`MINIMISE_ALL`, `RESTORE_WINDOWS`, `TILE_WINDOWS`, `NEXT_WINDOW`, `PREVIOUS_WINDOW`	Desktop window management.
`SWITCH_WORKSPACE`	Switch virtual desktop/workspace.
`OPEN_SYSTEM_SETTINGS`, `OPEN_AUDIO_SETTINGS`, `OPEN_POWER_SETTINGS`, `OPEN_FILE_SYSTEM`, `OPEN_COMMUNICATION_HUB`, `OPEN_MEDIA_HUB`, `OPEN_NETWORK_BROWSER`	Open specific OS setting panels / hub launchers.
`OPEN_IDE`, `OPEN_IN_IDE`	Open the IDE (optionally at a file).
`OPEN_BROWSER`, `OPEN_URL`	Open browser / a URL (local hand-off; respects net toggle for fetches).

3. Input & screen control

Action(s)	What it does
`KEYBOARD`	Send keypresses (`os_controller.press_key`).
`MOUSE_CONTROL`	Move/click the mouse (`os_controller.mouse_click`).
`VOLUME`	Get/set system volume.
`SET_CLIPBOARD`, `GET_CLIPBOARD`	Read/write the clipboard.
`SCREENSHOT`	Capture the screen.

4. Gaze (webcam eye-tracking)

Action(s)	What it does
`GAZE_ENABLE`, `GAZE_DISABLE`	Start/stop the MediaPipe gaze engine (writes `latest_gaze.json` @10Hz).
`GAZE_CALIBRATE`	Run gaze calibration.
`GAZE_CLICK`	Move the cursor to where you're looking and click.
`GAZE_STATUS`	Report gaze-engine state.

5. Media

Action(s)	What it does
`PLAY_MEDIA`	Play a song/playlist on a named platform (Spotify playlist-tab / YouTube Mix-radio); honest about reachability; explicit platform never falls through to a second.
`PAUSE_MEDIA`, `STOP_MEDIA`, `NEXT_MEDIA`, `PREVIOUS_MEDIA`, `REPEAT_MEDIA`, `SHUFFLE_MEDIA`	MPRIS/playerctl transport control.
`MEDIA_CONTROL`	Generic media-control dispatch.
`SKIP_YOUTUBE_AD`	Skip a YouTube ad. *(inferred)*

6. Files & documents

Action(s)	What it does
CREATE_FILE, CREATE_FOLDER, READ_FILE, LIST_DIR	Filesystem ops (path-allowlist gated).
SUMMARIZE_FILE	Summarise a file's contents (also the route for "read your persona.auto.txt / settings").
CONVERT_DOCUMENT	Convert document formats.
CREATE_DOCUMENT = CREATE_DOC = DOC_GENERATE = GENERATE_DOCUMENT = WRITE_DOCUMENT	Generate a document (the Report-Builder family).
ANALYZE_CSV, ANALYZE_PDF, ANALYZE_PDF_FOLDER, ANALYZE_IMAGE, OCR_IMAGE	Local file-type analysers (CSV stats, PDF text/structure, image VL description, OCR).

7. Vision & screen understanding

Action(s)	What it does
ANALYZE_IMAGE	Local GGUF vision-language description of an image.
OCR_IMAGE	Tesseract OCR text extraction.
AMBIENT_VISION	Toggle periodic background screen glances.
SCREEN_LOCATE	OCR-locate a named UI element or word on screen ("the button that says X", "find the word compose"), and click it ("click compose").
SCREEN_READ_ANALYZE	Screenshot → analyse what's on screen.
IMAGE_STATUS	Report vision/image-engine state.

8. Coding & code-repair

Action(s)	What it does
CODE_SOLVE	The frontier coding agent: plan → DAG decompose → UCB tree search → verify (syntax/exec/tests) → repair → bug-memory.
GENERATE_SCRIPT = CREATE_SCRIPT = WRITE_SCRIPT = GENERATE_CODE = WRITE_CODE	Generate a script — routes through the coding agent (falls back to inline gen).
GENERATE_PROJECT	Generate a multi-file project.
EXAMINE_CODE	Tiered file scan (syntax/import → lint → gated LLM logic review) → offer fix.
FIX_FILE, CONFIRM_CODE_FIX, CANCEL_CODE_FIX	Apply/confirm/cancel a verified auto-reverting code fix.
FILE_AUDIT, SHOW_DIFF, CODE_CHANGES	Audit a file / show a diff / report recent code changes.

9. Self-management & self-improvement

Action(s)	What it does
SELF_REPORT	Grounded runtime self-report (deterministic, from live state).
SELF_ANALYZE, SELF_IMPROVE, SELF_IMPROVEMENT_LOG	Analyse failures / surface improvement proposals / show the improvement log.
SELF_PATCH	Generate+apply a verified self-patch.

Action(s)	What it does
SELF_TEST	Run self-tests.
SELF_UPGRADE = SELF_UPDATE	Maintenance: git pull, pip, rebuild FAISS/KG, refresh manifest + system index.

10. Introspection & audits (grounded, deterministic)

Action(s)	What it does
RUNTIME_STATUS, RUNTIME_AUDIT	Live runtime report; RUNTIME_AUDIT also runs **health probes** (plugin mgr, memory, agent bus, habit integrity, recent failures).
GUI_RUNTIME_AUDIT, IMPORT_AUDIT, DIAGNOSE_WRAPPERS, RESOLVE_RUNTIME_PATHS	GUI/runtime audits, import smoke-audit, executor-wrapper diagnostic, path resolution.
FRONTIER_STATUS, ELI_IDENTITY_AUDIT	Full cross-system matrix / identity-classification audit.
COGNITION_STATUS, EXPLAIN_COGNITION_RUNTIME, EXPLAIN_MEMORY_RUNTIME	Cognition + memory runtime explainers (verbatim, deterministic).
EXPLAIN_ALL_REASONING_MODES, REASONING_MODE_STATUS	Describe the 5 reasoning modes / current mode.
EXPLAIN_LAST_RESPONSE, EXPLAIN_LAST_FAILURE, NAME_SOURCE_AUDIT, ROUTING_FAULT_EXPLAIN	Explain the last answer/failure, the source of your name, a routing fault, or how long the last turns took.
HARDWARE_PROFILE, GPU_STATUS, GET_STATUS, AWARENESS_STATUS	Hardware/GPU/general/awareness status.
HELP, LIST_CAPABILITIES	Help / list the capability surface.

11. Memory & identity

Action(s)	What it does
MEMORY_RECALL, MEMORY_STORE, MEMORY_STATS, MEMORY_STATUS	Recall/store memories; memory inventory/stats.
PERSONAL_MEMORY_SUMMARY, PERSONAL_MEMORY_DEEP_EXPLAIN	“What do you know about me” (clean) / deep memory-internals explain.
USER_IDENTITY_SUMMARY, USER_INFO_REPORT, REFRESH_USER_INFO	Who-you-are summary / full profile report / rebuild the living profile.
SET_USER_NAME	Set the user’s name.
MESSAGE_TIME_QUERY	“What time did I first/last message (today)” — from `conversation_turns`.
CLEAR_CHAT_HISTORY	Clear chat history.
PERSONA_LOCK_SET, PERSONA_LOCK_CLEAR, PERSONA_LOCK_STATUS	Lock/unlock/inspect the persona.

12. Habits, proactivity & goals

Action(s)	What it does
CONFIRM_HABIT, DECLINE_HABIT, HABIT_STATUS	Approve/decline a detected habit; report habits.
MORNING_REPORT	The consolidated morning brief (news digest + activity + attention).

Action(s)	What it does
PROACTIVE_START, PROACTIVE_STOP, PROACTIVE_STATUS	Control the proactive daemon.
EXECUTE_GOAL	Execute a mission-layer goal (governed).
RESUME_TASK	Explicit “continue/resume the X project” — goal_store.find_task() bypasses the passive 14-day idle window, states real staleness, says plainly when nothing matches rather than guessing.
BACKGROUND_JOBS, CHECK_JOB	List background tasks / check a job id.

13. Self-healing remediation (Linux)

Action(s)	What it does
PREPARE_REMEDIATION	Diagnose a failure + build a repair plan (e.g. install a missing app via apt/snap/flatpak).
CONFIRM_PENDING_REMEDIATION, CANCEL_PENDING_REMEDIATION	Execute / cancel the pending repair (sudo terminal, lock-handling, verification).
CHECK_TARGET_STATUS	Check whether a target app/path is now present.

14. Voice & transcription

Action(s)	What it does
DICTATE, TRANSCRIBE	Dictation / transcribe audio (faster-whisper).
LISTEN_FOR_COMMAND	One-shot listen.
STT_DIAGNOSTICS, VOICE_DIAGNOSTICS	Diagnose the STT/voice pipeline (mic, ducking, wake gate).
SAY / SPEAK *(plugin: tts)*	Speak text (Piper TTS).

15. Time, timers & utility

Action(s)	What it does
TIME = GET_TIME, DATE = GET_DATE	Current time / date.
SET_TIMER, SET_ALARM	Timers/alarms. *(inferred)*
CHECK_CHRONAL_ALIGNMENT	Easter-egg: “Chronal alignment nominal. Local time: …” (a playful time report).
DATA_FABRICATOR	Generate a document from a topic (via CREATE_DOCUMENT) **and open it in an editor** (code/gedit/kate/nano).
RUN_CMD, SHELL_EXEC	Run a shell command — **fail-closed** (blocked unless ELI_ALLOWED_CMDS is set or ELI Full Control is on).

16. Plugins (management)

Action(s)	What it does
PLUGIN_LIST, PLUGIN_SEARCH, PLUGIN_STATUS	List installed / search registry / status.
PLUGIN_INSTALL, PLUGIN_UNINSTALL, PLUGIN_ENABLE, PLUGIN_DISABLE	Install/remove/enable/disable a plugin.

17. Plugin-backed capabilities

Action	Plugin	What it does
WEB_SEARCH	web	DuckDuckGo/SearXNG web search (toggle-gated).
GET_WEATHER	weather	Local geocode + open-meteo (toggle-gated).
NEW_NOTE/WRITE_NOTE, LIST_NOTES, SEARCH_NOTES	notes	Markdown notes with FTS.
ADD_EVENT, LIST_EVENTS, REMOVE_EVENT	calendar	ELI's own local calendar and reminders (runtime/agenda.py): add, move, list, remove. Mirrored to calendar.ics.
POMODORO_START, POMODORO_STOP, POMODORO_STATUS	pomodoro	Focus timers.
CPU_USAGE, RAM_USAGE, SYSTEM_STATS	system_stats	CPU/RAM/disk/network.
SMART_HOME	executor and device server	Smart-home control via ELI's own MQTT device server (ESPHome / Tasmota / Zigbee2MQTT); rooms, scenes, real automations. Home Assistant removed.
SPEAK	tts	Text-to-speech.
NEWS_FETCH	(executor + news tool)	Fetch + synthesise news (rolling 3-hourly reflections → 24h digest).

(Also reachable: web_automation plugin — Playwright browser navigate/search/screenshot; document_reader plugin — PDF/docx read+index.)

Reading note

Alias families that collapse the count: app-open (3), app-close (4), minimise (4 across window/app + UK/US), document-generate (5), script/code-generate (5), time/date (2 each), self-upgrade/update (2). Removing pure aliases yields **~110 genuinely distinct capabilities** — still an unusually broad surface for a local single-user assistant, spanning OS control, media, files, vision, gaze, voice, coding, memory, introspection, autonomy, remediation, and plugins.

Part 2 — runtime/ module catalogue (94 files, 36.4k lines)

The largest package. It's the **grounding/governance + introspection + plumbing** layer that wraps the probabilistic model. Grouped by function:

Grounding & anti-confabulation

Module	LOC	Role
`deterministic_grounding_gate.py`	3385	Renders control/status answers directly from live runtime (bypasses the model). One ordered `render_action` pipeline of layers composed by `_stack`; an unhandled action returns a `missing_deterministic_renderer` surface.
`grounding_escalation.py`	698	When a **checkable factual** question is poorly grounded by the bus, escalates through agent tiers instead of letting the model confabulate (the "Orlo's real name" failure class).
`diagnostic_patterns.py`	112	Regexes that catch vague/dynamic status confabulation ("currently processing updates…") and image-status fabrication.
`control_contracts.py`	1242	Control-action evidence contract: build evidence → validate the model's output doesn't violate it → finalise.
`persistence_gate.py`	204	Gates what gets stored — refuses to persist internal dumps / error-pattern noise as memory.

Self-honesty / introspection / self-reporting

Module	LOC	Role
`live_introspection.py`	736	Live runtime snapshot, last trace, stored user name, mines user-fact candidates, agents-for-action, build_report.
`deterministic_introspection.py`	591	The engine's live diagnostic dispatcher (`handle_diagnostic_action`) — deterministic answers for RUNTIME_STATUS / EXPLAIN_* / IMPORT_AUDIT.
`truth_report.py`	436	Runtime truth report (git, nvidia, GGUF runtime, import health).
`frontier_status.py`	433	Full cross-system status matrix (runtime/memory/awareness/proactive/image/worl
`eli_identity_audit.py`	416	Identity-classification audit (source inventory, contract counts, capability matrix).

Module	LOC	Role
`reasoning_status.py`	199	Current reasoning-mode reporting.
`experimental_inventory.py`	146	Inventory of experimental projects.

Self-improvement & code-awareness

Module	LOC	Role
`self_improvement.py`	1852	Failure logging/clustering, patch generate→verify→apply→**auto-revert**, full patch cycle, plugin-stub gen.
`code_examiner.py`	910	Tiered file error scan (syntax/import → lint → gated LLM) → offer → verified fix.
`code_monitor.py`	261	Detects source changes via git diff, classifies by subsystem, summarises for memory/context (ELI is aware of its own code changes).
`capability_sync.py`	419	**AST-discovers** the live capability surface, diffs, writes `capability_manifest.json` — this is why the count is *measured*, not asserted.
`generated_script_guard.py`	1084	Validates LLM-generated scripts, **quarantines invalid ones**, and ships vetted canned scripts for known patterns (GPU-watch, redshift, etc.).

Self-healing remediation

Module	LOC	Role
`grounded_remediation.py`	1717	Diagnoses failures (missing app/path/browser) → builds an apt/snap/flatpak repair plan → offers → **executes** (sudo terminal, lock-handling, verify).
`incident_log.py`	21	Writes incident records.

Awareness & boot

Module	LOC	Role
`awareness_boot.py`	367	Boots all awareness subsystems at startup, returns an `AwarenessState` the engine queries.
`action_commitment.py`	220	Detects when ELI's reply commits to an action. The engine then runs it only if it is a read (news, weather, calendar listing) or something the user's own message asked for; anything that changes state waits for a "yes".

Autonomy / operator (governed)

Module	LOC	Role
`operator_state.py`, `operator_feed.py`	~177	Operator console state: proposals, goals, self-model status, event feed.
`pending_proposal.py`	577	What ELI's last reply left open, for 30 minutes: the steps it offered (`read_reply`: actions, tasks ELI does by writing, explicit-only), the question it ended on, a detail an action asked for. Reads the user's reply against it: `is_consent`, `chosen_items`, `selection`, `is_the_detail`.
`agenda.py`	1051	ELI's calendar and reminders: SQLite store, plain-language dates and times (`parse_when`), a notifier thread that delivers what is due (and what was missed while shut), `calendar.ics` mirror, the next 36 hours as a prompt line.
`approval_engine.py`	103	Who may propose / evaluate a proposal record (governance).

Response surfaces & governance

Module	LOC	Role
`user_visible_response_surface.py`	367	Installs the engine's user-visible response surface (runtime/identity/name-source formatting + streaming coercion).

Module	LOC	Role
`visible_output.py`, `visible_text.py`	~150	Central visible-output contract; stringify/sanitise streamed output.
`response_policy.py`, `response_contracts.py`	~160	Classify response mode; per-action contracts.
`final_response_provider.py`	~85	Assemble the final prompt; per-action generation decoration; fastpath context.

Personal-memory surfaces

Module	LOC	Role
`personal_memory_surface.py`	452	“Is this a personal-memory query” + surface builder.
`personal_memory_clean_response.py`	338	Clean “what do you know about me” report (reset-aware, poison-filtered, dynamic-fact aging).
`personal_memory_deep_response.py`	450	Deep memory-internals explain (schema/tables/functions) + routing-fault explain.
`profile_extractor.py`	1667	Extracts user facts from turns (role/interests/field/“remember that I…”), writes user_patterns + LLM session summaries; recency refresh. `_SINGLE_VALUED_PATTERNS` (single source of truth, `user_model.py` imports it) gets belief-weighed supersession with revision history on a correction, not accumulation of contradictory rows.
`identity_validation.py`	176	Validate identity candidates.

Typed pipeline plumbing (evidence/packets)

Module	LOC	Role
`evidence_ledger.py`	679	Records artifacts/events with signatures; recent generated artifacts; status evidence.
`evidence_arbitration.py`	212	Scores competing evidence (stage packets + tool results + goals), dedup-by-fingerprint, keep-max.
`stage_packet_store.py`, `stage_packets.py`, `pipeline_models.py`	~177	The typed packet substrate: route/plan/evidence/generation/output packets flow between stages; this is the plumbing behind “no fake actions”.

Module	LOC	Role
`background_tasks.py`	252	In-process multi-threaded task manager (heavy work → job id → `CHECK_JOB`).
`runtime_policy.py`	89	Per-turn budgets/timeouts/context size from runtime snapshot.

Security

Module	LOC	Role
`security.py`	210	`SecurityManager` — fail-closed command/path/app allowlist sandbox.

Part 3 — `cognition/` module catalogue (47 files, 20.8k lines)

The thinking layer: agents, orchestration, inference, persona, reasoning, governance.

Orchestration & agents

Module	LOC	Role
`agent_bus.py`	3517	15 specialist agents on a dependency DAG (topological layers) + calibrated weight-free confidence aggregation + per-action agent selection.
`orchestrator.py`	1123	Gradient 12-stage pipeline (all CHAT modes): planner → `retrieve_for_turn()` → `dispatch_specialists()` → heuristic rerank → context assembly. Composes the specialist bus; no longer Quick-only bypass.
`learning_coordinator.py`	82	Stage 12 `finalize_turn()` — store assistant turn, publish meta, `_learn_from_result()`. The latter is gated by `source` (`_is_autonomous_source()`): a habit-scheduler-fired turn doesn't feed `habit_events`, so ELI's own automation can't reinforce itself as fresh evidence of a user routine.
`hyde.py`	69	Hypothetical-document-embedding query expansion.

Module	LOC	Role
`reranker.py`	178	Candidate reranking (token overlap + source priority).
`introspection_agent.py`	164	Wraps introspection for the bus (pipeline/memory/runtime/audit).
`llm_intent.py`	260	LLM intent parsing fallback (GGUF, cached).

Inference

Module	LOC	Role
`gguf_inference.py`	3266	Model-agnostic GGUF inference: model resolution (no baked model), family-aware chat templating, graceful GPU-layer fallback, streaming, output cleaning, token budgeting.
`inference_broker.py`	222	Thin GGUF broker (`infer`) used by agents/coding/patching.

Reasoning & engagement

Module	LOC	Role
`reasoning_modes.py`	671	The 5 modes — canonicalisation, per-mode private system instruction, execution contract (samples/branches/stages + dynamic token budget), reasoning-leak stripping.
`engagement_tracker.py`	249	Session depth tracking → **auto-escalates reasoning mode** (Quick→Normal→Advanced→Research) as a conversation deepens; session narrative.
`working_memory.py`	387	Turn-scoped pinned facts (pin/absorb/evict/persist/restore).

Context & grounding

Module	LOC	Role
`context_synthesiser.py`	776	Builds the precise prompt context: persona handoff, turns block, vector block, live-runtime brief, budgeting.
`grounded_status.py`	648	Identity + memory-inventory rendered directly from profile/DBs (direct grounded answers).

Persona (the living voice)

Module	LOC	Role
`persona.py`	375	Canonical persona authority — base + auto sections, preferences, compose/refresh.
`persona_updater.py`	748	Re-derives the persona overlay from memory/reflection/habits/runtime patterns; KG population; stale-fact aging.
`persona_hygiene.py`	130	Cleans/dedups/prunes the auto-persona.
`persona_status.py`, `persona_values.py`	~108	Persona status report; values store.

Output governance (consolidated this session)

Module	LOC	Role
`output_governor.py`	1694	**Canonical** governance: sanitize, role-prefix/identity-drift repair, self↔user confusion repair, style cleanup, confabulation detection, quality scoring, memory-worthiness, GGUF-artifact cleaning (`clean_gguf_artifacts`), evidence validation.
`response_governance.py`, `response_sanitizer.py`	28+14	**Re-export shims** → output_governor (kept for back-compat).
`tone_analyzer.py`	371	Analyses recent user turns → tone preferences ELI adapts to over time.

Profile

Module	LOC	Role
`user_info_builder.py`	752	The living, versioned user profile: multi-source gather, noise filter, categorise, hash, diff-on-change.

Part 4 — remaining packages (module catalogue)

`kernel/` (the engine + boot) — 17.5k lines

Module	LOC	Role
`engine.py`	16014	`CognitiveEngine` — the conductor: persona, generation settings, the 5 `_run_*` reasoning passes, grounding overrides, synthesis prompt build + the context-bloat cap, fragment/placeholder guards, dispatch gate to bus vs orchestrator, startup loops (reflection/habit/scheduler/self-improve/proactive).
`world_model.py`	278	Symbolic self-model: Identity/Runtime/Memory/Goal/Capability states + snapshot/merge.
`pipeline_trace.py`	85	Canonical S01–S12 stage names and `log_pipeline_stage()` observability.
`state.py`	371	User/runtime state + profile (active user id, name, profile text).
`self_upgrade.py`	575	Self-upgrade orchestrator (git pull, pip, rebuild FAISS/KG, manifest, system index).
`scheduler.py`	74	Kernel thread-pool/timer scheduler (generic).
`pipeline.py`	148	Pipeline step description.

`core/` (paths, settings, hardware, safety) — 11.2k lines

Module	LOC	Role
`runtime_settings.py`	1290	Canonical settings.json load/save, legacy-key migration, portable-path healing/sanitising.
`hardware_profile.py`	2818	Auto-detect GPU/VRAM/RAM → fit ctx/gpu_layers/batch (KV-cache + compute-buffer reserve); model discovery; hardware authority enforcement.
`paths.py`	768	Single-source-of-truth path resolution (data/config/cache/db/models/voices…).
`startup_hardware_optimizer.py`	877	Boot-time hardware optimiser (GPU select, layer/ctx allocation, mode presets).
`config.py`	357	Thin config shim over runtime_settings (canonical key mapping).
`model_download.py`	673	Curated GGUF catalogue + VRAM-based recommend + download.

Module	LOC	Role
`dag.py`	470	**Generic DAG engine** (Kahn topo-order, parallel layers, critical path) — shared by the agent bus + coding engine.
`netguard.py`	586	**Offline-by-default** socket-level network gate (fail-closed) + `guarded_urlopen`/`http_get_json`.
`memory_reset.py`	333	Factory-reset of memory/identity (scrub names, clear DBs, backup).
`cognition_tunables.py`	273	User-tunable knowledge-gathering limits + synthesis cap registry (GUI-surfaced).
`grounding.py`	145	`is_grounded_query` classifier.
`crisis_guard.py`	113	STT-robust self-harm detector + persona steering directive.
`portable_paths.py`, `db_paths.py`, `legacy_paths.py`	small	path helpers.

`memory/` — 8.8k lines

Module	LOC	Role
`memory.py`	6003	The `Memory` class (85 public methods): semantic store/recall, conversations, habits, failures/improvements, storage policy, upkeep, weight decay, KG bridge, `mark_failure_resolved`, `disable_invalid_habit_rules`.
`knowledge_graph.py`	643	Entity/relation graph + multi-hop BFS (`related`) + `context_for_prompt` + extract-from-memory.
`retrieval.py`	253	Shared turn retrieval (`retrieve_for_turn`) with an 8 s cache, time window and `memory_diag` stats.
`policy.py`	118	Storage policy: origin, dedupe key, forgetting curve, archive rule.
`claims.py`	266	Bitemporal claims about the user: valid interval, learned-at, supersession, extraction.
`unified_retrieval.py`	168	Orchestrator stages consume `retrieve_for_turn` through it.
`vector_store.py`	637	FAISS index (L2, 1/(1+dist) sim) + nomic embedder + keyword fallback + auto-rebuild.

Module	LOC	Role
system_index.py	278	OS app/exe/dir index (the launcher backing — your machine's executables, thousands, indexed live per machine).
memory_truth.py, memory_adapter.py	small	inspection/compat/session helpers.

perception/ — 10.2k lines

Module	LOC	Role
audio_stt.py	2472	faster-whisper STT + VoiceGate (wake-word, debounce, incomplete-command wait), self-echo suppression, output ducking, per-user voice profile bias.
vision.py	698	Local GGUF VL (Moondream fast / primary), hot-swap with text model, CPU-pinned CLIP, OCR.
tts_router.py	1296	Piper/espeak TTS, voice selection, unspeakable-fragment guard; plays at the device's own rate when it refuses the voice's.
audio_samples.py	74	A sound file as one channel of float32 samples at the rate a consumer needs (speech input, playback, the wake word, the voice profile): PCM WAV read with the standard library, other formats with soundfile.
os_controller.py	572	Screenshot/volume/keys/mouse/clipboard + gaze_click.
screen_locator.py	497	OCR (tesseract) → locate a named UI element on screen.
gaze_engine.py	358	MediaPipe face-gaze + calibration mapper + One-Euro smoothing → latest_gaze.json @10Hz.
local_whisper_stt.py, ambient_vision.py, analyze_{pdfs,image,mesh,csv}.py, log_rotation.py, voice_worker*.py	~1.2k	Whisper backend; ambient glances; file analysers; log rotation; voice workers.

planning/ (autonomy, habits, proactive) — 4.3k lines

Module	LOC	Role
`proactive_daemon.py`	1523	Background 10-min loop: pattern/code analysis, habit detect+offer, morning report, error tracking.
`autonomy_scheduler.py`	334	Policy-gated (observe/proposal/goal-driven) goal scheduler w/ cooldown + attention queue.
`habits.py`	370	Habit detection (circular time-of-day clustering, shift and lapse notes), offer/pending state, disabled-by-default.
`routine_stats.py`	125	Circular clustering, routine shift and lapse detection.
`habits_scheduler.py`	170	Fires active habits at their time (self-heals legacy rows, once-per-minute dedupe).
`goal_store.py`, `goal_models.py`, `goal_tick.py`, `operator_goal_actions.py`	~400	Mission goals (priority/cadence/risk/constraints/success-criteria) → governed proposals.
`attention_queue.py`, `proposal_queue.py`, `proposal_*.py`, `jobqueue*.py`, `autonomy_controller.py`	~900	Attention ranking; proposal queue/archive; job queue; safe autonomy ticks.

`coding/` (the frontier coder) — 2.1k lines

Module	LOC	Role
`agent.py`	215	`CodeAgent.solve` — composes the loop; broker-backed generator; full provenance.
`bug_memory.py`	340	Bug classification + (signature→fix) long-term SQLite memory w/ fuzzy recall.
`verification.py`	353	Gate ladder (syntax→exec→tests) + weighted scoring + test synthesis.
`sandbox.py`	234	Bounded isolated execution (scrubbed env, CPU limit, timeout).
`search.py`	176	UCB1 tree search over a temperature-ladder beam.
`plan_graph.py`	164	Subtask DAG decompose + topological solve + compose.
`planner.py`, `cost.py`	~210	Planner/implementer split; cost → foreground/background decision.

`learning/` (real LoRA self-training) — 4.3k lines

Module	LOC	Role
`lora_trainer.py`	834	Real torch/PEFT/transformers `Trainer` LoRA run.
`lora_trainer_guard.py`	630	`TrainerTarget` plans + safety validation.
`dataset_builder.py`	552	Build supervised dataset from logged turns/corrections (PII-redacted, deduped).
`lora_eval.py`	485	Adapter eval suite.
`bootstrap_phi3_base.py`, `base_model_resolver.py`, `dataset_filters.py`, `export_trainable_dataset.py`, `merge_reviewed_datasets.py`, `training_preflight.py`	~1.4k	Trainable base download/resolve, quality gates, export/merge, preflight.

`contracts/`, `integrations/`, `system/`, `utils/`, `cli/`

Module	LOC	Role
`contracts/runtime_status.py`	571	Canonical runtime-status evidence contract (detect question, build live evidence, validate/repair).
`contracts/grounded_control.py`	216	Grounded-control synthesis guard (evidence-complete? suppress bad clarifications).
`integrations/mpris/playerctl_backend.py`	771	MPRIS2/playerctl media backend (player resolve, transport, status).
`integrations/ollama/client.py`	322	Optional Ollama backend client.
`system/portable_app_control.py`	742	Cross-platform installed-app discovery + open/close/minimise.
`utils/platform_compat.py`	1666	Cross-platform layer (open url/file/app, volume, clipboard, notify, executables).
`utils/log.py`, `cli/headless.py`	~200	Central logger; headless terminal REPL.

`world/` (embodied self-model) — 1.8k lines

Module	LOC	Role
`agency/autonomy_engine.py`	260	Awareness vector (easing/decay) → avatar-room routing + symbolic-object creation during reasoning; provenance/snapshots/journal.
`agency/{policy,goal_ecology,habit_engine,reflection_bridge,world_commitments}.py`	~1.1k	Permission classes; goal decay; default habits; reflection→world; world identity.

Module	LOC	Role
`core/schemas.py`, `core/ontology.py`	~160	World state schemas (rooms/objects/events/actions/awareness); object templates.
`renderers/pyside6/{world_scene,world_panel}.py`	~1105	The PySide6 “Eli’s World” tab (house/rooms/avatar/objects, live state).
`avatar/*`, `persistence/*`, `world_event_bus.py`, `local_world_bridge.py`	~700	Avatar behaviour/locomotion/persona-map; jour-nal/provenance/snapshots/storage; event bus + bridge.

`gui/` (PySide6 desktop) — 27.4k lines

Module	LOC	Role
`eli_pro_audio_gui_v2_0.py`	13149	Main window: 13 top-level tabs + adapters (CentralMem-ory/LocalModel/Ollama/Executor bridges, the `_GUIEngineAdapter`), chat, drag-drop, reasoning-mode auto-select, all toggles.
`labs_tab.py`	5744	Labs workspace: Notebook, Memory browser, Jupyter launcher, Calculator(+constants), Physics tables, **Report Builder** (evidence-grounded docs), File-Chat, Workspaces, Sim-IDE.
`app.py`	827	Launcher / first-boot auto-tune / `main()`.
`panels/startup.py`	1974	StartupModelSelectionDialog, FirstBootWizard, HardwareTuningDock.
`panels/settings.py`	825	Advanced Settings (Agents/Models/**Cognition**/Plugins/Self-Upgrade).
`docks/operator_console_dock.py`	303	Governed operator console (pro-posals/goals/policy/attention).
`widgets/`, `tabs/`, `panels/agent_wizard.py`, `docks/proactive_dock.py`, `qt_compat.py`	~1k	Ollama selector, experimental/world tabs, agent wizard, proactive dock, Qt shim.

`plugins/` — 10 built-ins + manager

Module	Role
`manager.py` (708L)	Discover/install/enable/disable/execute; auto-load; builtin-stub gen; registry.

Module	Role
`web`, `web_automation`, `weather`, `calendar`, `notes`, `pomodoro`, `system_stats`, `media`, `tts`, `document_reader`	The 10 built-in plugins (see Part 1 §17).
`base/base.py`	Plugin base class + action validation + loader.

Part 7 — modules not covered above

Every remaining module under `eli/`, with its line count and a one-line role (the module's own docstring where it has one). Regenerate the counts with the snippet in `tools/refresh_doc_metrics.py`.

`runtime/`

Module	Lines	Role
`active_project.py`	78	Active project signal.
`api_users.py`	145	API users + roles (admin / member RBAC) for the web server.
`auth.py`	4	Tombstone comment: the unfinished multi-user AuthManager was removed; authority lives in `authority_gate.py`, `security.py` and the engine's persona lock.
`authority_gate.py`	49	authority_gate — intentional stub.
`autopilot_debugger.py`	377	Autopilot debugger — one loop that turns a failure into a plan.
`ble_light.py`	247	BLE light control — the GATT writes ELI never had.
`bt_platform.py`	1335	Cross-platform Bluetooth radio detection, recovery, and classic discovery.
`codebase_graph.py`	306	Codebase self-graph — ELI's live, grounded model of how its OWN code connects.
`command_splitter.py`	208	Split one utterance that chains several commands into its parts; read a message as sentences and tell an instruction from a statement or a question. Holds the one list of verbs a command starts with.
`conversation_thread.py`	284	Conversation thread awareness — topic carryover, proactive grounding, web queries.

Module	Lines	Role
desktop_launchers.py	718	Redistribution-safe desktop / Start Menu launchers.
deterministic_failure_patches.py	108	Rule-based code patches for recurring executor failures without LLM guessing.
device_drivers.py	1124	Pluggable LOCAL-control drivers for ELI's device server — control devices that don't
device_names.py	219	User-chosen device names — stable keys for voice control across hardware/OS.
device_server.py	1453	ELI's own device server — original, MQTT-first. No Home Assistant.
eval_review.py	117	Eval + LLM-judge board, in the shape Labs ▸ Test & Review already renders.
evidence_planner.py	302	Evidence planner + gatherer — the DAG/plan principle for generative & grounded tasks.
failure_taxonomy.py	230	Classify a runtime failure by what actually went wrong, and where.
gguf_runtime_report.py	257	Portable GGUF / inference diagnostics — model-agnostic, path-agnostic.
home_intel.py	114	ELI home intelligence — weave the LLM into the home/device system.
home_mesh.py	435	ELI home mesh — tiered brains with LAN failover.
inference_footprint.py	413	Live inference RAM — read from the loaded llama.cpp model and this process, never guessed.
last_trace.py	58	Persists the last response trace, keyed to the process id so a restart never reports a closed session's turn.
license_info.py	99	Locate and print ELI's licence, wherever ELI happens to be running from.
local_connectivity.py	764	Local-only WiFi and audio routing — sovereign stack, no cloud.
memory_provenance.py	153	Memory provenance and verification tier for grounded recall.
mqtt_setup.py	444	Cross-platform MQTT broker onboarding for ELI redistribution.

Module	Lines	Role
native_locks.py	14	Process-wide locks for native llama.cpp and FAISS entry points, preventing cross-context heap corruption.
proposal_adapters.py	1	Re-export shim of planning/proposal_adapters.py.
reflection.py	377	Reflection engine — analyses memories, conversations, and patterns to extract insights.
lessons.py	345	Expiring, checked lessons from recurring failures (trigger, evidence, predicted outcome, retire when unhelpful).
relational_facts.py	83	Extract relational facts the user mentions in passing.
repair_playbook.py	243	ELI repair playbook — decision guide for self-maintenance, code examine, and upgrades.
report_pipeline.py	198	Multi-stage grounded document pipeline (Report-Builder discipline, chat scale).
research_corpus.py	439	Local research corpus workspaces.
scheduled_tasks.py	569	Scheduled / overnight advanced tasks.
self_facts.py	333	Verified facts about ELI's own construction, assembled from live sources.
self_maintenance.py	175	Unified self-maintenance orchestration for ELI.
self_maintenance_config.py	6	Shared constants for ELI self-maintenance (analysis windows, cluster gates).
self_model_refresh.py	61	refresh_all_overlays_nonfatal: refreshes the persona and user-profile overlays without raising.
self_status.py	264	Real, measured self-status for ELI.
server_util.py	240	Helpers for detecting an existing ELI web server and firewall guidance.
session_continuity.py	100	Session continuity — live thread memory injected into every CHAT turn.
shell_followup.py	68	Resolve deictic shell follow-ups ("run that command") from prior assistant output.

Module	Lines	Role
site_address.py	119	The address behind a site's name or a reference to one ("nasa website", "that website"): from the conversation, then a web search. Never built from the words.
state_providers.py	80	State providers — the hook that lets components expose save/restore state to a
test_generator.py	283	ELI-assisted behavioural test generation (Phase 4).
test_review.py	234	Full test/project run → review workflow.
user_model.py	372	Continuous User Model — ELI's living, semantic, auto-updating model of the user.
voice_assets.py	558	Ensure the local voice models (STT + TTS weights) are present.

cognition/

Module	Lines	Role
agent_spec.py	436	A real specification for a custom agent — objective, prompt, triggers, measures.
agent_trust.py	262	Trust for custom agent code — provenance, not just a hash in a dict.
belief.py	239	Belief revision — what ELI holds, how strongly, and what it takes to change it.
chat_grounding_gate.py	157	Fail-closed CHAT gate — skip LLM when user-fact grounding is required but absent.
context_budget.py	128	Sizing and trimming the memory block that goes into a prompt.
correction_patterns.py	224	Shared patterns for user correction / dispute turns.
emotion_palette.py	254	Emotion / tone palette — the shared taxonomy ELI expresses through.
emotion_timeline.py	452	Emotion timeline — the durable record of how the USER has been feeling.

Module	Lines	Role
`evidence_format.py`	196	Dates on evidence lines, from each row's own timestamp; `time_facts` (clock times in the user's message worked out against now); `this_conversation` (the dialogue block holds this conversation, not turns from days ago).
`expression_state.py`	63	Live avatar/expression state — the thin bridge that lets ELI's face react in
`memory_diag.py`	73	What memory retrieval actually did this turn, from telemetry, so ELI never has to guess why.
`model_identity.py`	237	Model-agnostic identity: chat family + thinking from GGUF metadata.
`model_load_diagnostics.py`	633	Say why a GGUF would not load — for ANY model, not a known list of them.
`model_output_tokens.py`	301	Canonical special tokens, stop sequences, and persona drift patterns for all GGUF families.
`personal_context_gate.py`	118	Gate when past-session plans/projects may enter the persona handoff.
`query_planner.py`	161	Deterministic query planning: the time window a question asks about.
`scoring.py`	116	Canonical scoring + confidence primitives for ELI.
`stance_capture.py`	151	Notice when ELI has taken a position, so it survives the conversation.
`stance_store.py`	288	Persistence for beliefs — ELI's own positions, and the record of revisions.
`tone_adaptor.py`	224	Tone adaptor — decides the emotion ELI *expresses* and feeds every output channel.
`turn_dossier.py`	404	Turn dossier — single assembly point for awareness, memory, and cognition context.
`user_claim_validator.py`	258	Post-generation validation of user-attribution claims in CHAT output.

`execution/`

Module	Lines	Role
app_aliases.py	90	Speech-damage layer for app names, on top of the cross-platform resolver.
command_dependency_graph.py	60	MULTI_COMMAND's infer_dependencies() — one optional LLM call for dependency edges among an already-split command list, validated via eli.core.dag.
command_sequence_log.py	102	MULTI_COMMAND's durable per-step log — crash-survival and skip-already-done on resubmit.
effectors/system_helpers.py	175	Cross-platform system helpers shared by executor and effectors (v3).
execution_planner.py	111	Builds RouteDecision and the typed ExecutionPlan/PlanStep artifacts — .agent_profile (agent-name filter) is the only field anything reads; .steps is never executed, only rendered as decorative text by EXECUTE_GOAL.
executor_enhanced.py	16035	The flat if a == "ACTION": dispatch every routed action runs through.
media_runtime.py	709	Media runtime control for ELI.
operator_actions.py	105	Valid states and helpers for operator (proposal) actions: pending, approved, rejected, blocked, pending_confirmation.
operator_policy.py	65	The valid operator policy modes: proposal_only, operator_supervised, goal_driven, observe_only. Only observe_only (silences the autonomy loop) and goal_driven (elevated + self-upgrade scheduling) have distinct behavior — proposal_only/operator_supervised are currently identical code paths.

Module	Lines	Role
`portable_intent_contract.py`	750	Portable intent rules; hard-blocks document, code and analysis prompts from `PLAY_MEDIA`. The one reading of an “open …” target that is a website (`web_open_target`) and of the window a command names (`window_target`).
`route_contracts.py`	219	Small predicates that classify a request for the router; whether an action changes anything (`changes_something`) and whether a sentence tells ELI to do what was matched in it (`fits`).
`router_enhanced.py`	8855	The router: `route(text)` regex-first priority pipeline with LLM-intent fallback (see `architecture.md` §4).
`shell_gate.py`	110	Centralised shell-command safety gate.

core/

Module	Lines	Role
`confidence.py`	9	The one confidence-score to label mapping.
`full_control.py`	46	ELI Full Control — a single master override that lifts ELI’s safety barriers.
`gpu_pack_runtime.py`	83	Activate the optional GPU pack before llama_cpp is imported (AppImage/portable).
`init_data.py`	283	First-run data initialiser — build ELI’s FULL database architecture up front.
`llama_cpu_compat.py`	240	CPU compatibility for llama-cpp-python — parity with GPU probe/fallback policy.
`load_plan.py`	975	What a model loads with on this machine, measured by llama.cpp’s load simulation on every GPU.
`model_tier.py`	141	Model-capability tier — the single signal that lets ELI’s cognition budgets
`secure_io.py`	94	Race-free writing of files that must never be world-readable.

Module	Lines	Role
`self_provenance.py`	207	Is this row ELI's own bookkeeping, or evidence about the user?
`sqlite_util.py`	99	Shared SQLite pragmas — WAL with a portable-safe fallback.
`toml_util.py`	19	Load TOML on Python 3.10+ (tomli backport) and 3.11+ (stdlib tomllib).
`worldclock.py`	348	Place-aware wall-clock answers.

`memory/`

Module	Lines	Role
`unified_retrieval.py`	168	Unified memory retrieval for orchestrator and agent bus.

`perception/`

Module	Lines	Role
`desktop_capabilities.py`	186	Cross-OS desktop control capability probe — one place for per-platform truth.
`mic_resolver.py`	613	ELI microphone resolver — pick a capture device that actually delivers audio,
`screen_analysis.py`	194	Local screen analysis — depth modes, memory recall, research context.
`tts_xtts.py`	394	Voice cloning backend (Coqui XTTS-v2).
`ui_ground.py`	322	Optional UI grounding backends — 100% local by default.
`ui_tree.py`	353	Accessibility-tree UI targeting (Linux / AT-SPI).
`voice_fx.py`	252	Character-voice effects layer.
`voice_profile.py`	417	Voice profile + prosody — the foundation for tone/emotion detection.
`voice_worker_streaming.py`	61	Stream-aware voice worker for ELI.
`wakeword.py`	456	Self-trained, fully-local wake-word detector (openWakeWord features + a custom head).

`planning/`

Module	Lines	Role
goal_autogenesis.py	224	eli.planning.goal_autogenesis
insight_synthesis.py	135	Cached, background-synthesised reflection insight.
jobqueue.py	184	Durable SQLite-backed job queue for external subprocess jobs; files under <data>/jobs.
jobqueue_cli.py	59	Durable background jobs that run as separate processes and survive a restart.
proposal_adapters.py	165	Adapters that make proposal execution results look like subpro-cess.CompletedProcess.
proposal_memory_bridge.py	64	Bridge that records proposal outcomes into memory through safe calls.
proposal_models.py	48	ProposalRecord: kind, payload, source, priority.

coding/

Module	Lines	Role
code_mode.py	91	Code-mode — the model writes a small program against ELI's own capabilities.
repo_context.py	91	Repo-context retrieval for the coding agent (Advancement C).
restricted_exec.py	170	In-process restricted execution for code-mode.

learning/

Module	Lines	Role
lora_pipeline.py	149	LoRA training pipeline — the DAG that chains the standalone learning stages.
review_queue.py	270	The human review gate that had no way to be passed.
target_registry.py	266	User-declared LoRA training targets — the registry that replaces the Phi-3 lock.

integrations/

Module	Lines	Role
`local_gguf/inference.py`	2	Compatibility shim: canonical GGUF inference lives in eli.cognition.gguf_inference.
`mcp/server.py`	194	MCP server — ELI's own capabilities exposed to any MCP client.
`media/capabilities.py`	181	Runtime media capability probe — what works on THIS machine/OS.
`media/cross_platform.py`	590	Cross-platform Spotify control and mpv IPC.
`media/media_deps.py`	141	Discover media CLI tools — bundled install root, then system PATH.
`media/spotify_intent.py`	139	Parse user phrasing into Spotify playback intents (no Web API).
`media/youtube_playback.py`	339	Shared YouTube playback helpers — mpv, yt-dlp clients, browser URLs.

system/

Module	Lines	Role
`process_guard.py`	447	Verified process-kill safety for CLOSE_APP and friends.

utils/

Module	Lines	Role
`jsonio.py`	40	Small tolerant JSON readers shared across the runtime.
`native_io.py`	74	Silence stderr written by C extensions, at the file-descriptor level.

onboarding/

Module	Lines	Role
`interview.py`	701	Light, skippable first-run onboarding interview.

setup/

Module	Lines	Role
`__main__.py`	48	Entry: python -m eli.setup [–wizard] [–launch] [–status] [–full-install]
`hardware_policy.py`	330	Install-time accelerator inventory and CPU-only recommendation.
`install_backend.py`	185	Cross-platform install backend — streams install.sh / install.ps1 into GUI progress.
`install_messages.py`	116	Curated installer copy — ELI voice: direct, dry, nerdy, occasionally a pisstaker.
`platform_profile.py`	142	Install-time platform detection — routes GUI, headless, and Android profiles.
`status.py`	120	Read-only setup completeness checks for first-run / grandparent setup.
`unified_installer.py`	686	One-click GUI installer — backend terminal work with OS-style progress and ELI wit.
`wizard.py`	298	Grandparent-ready graphical setup wizard — every first-run stage in one place.

world/

Module	Lines	Role
`avatar/behaviour_controller.py`	36	`AvatarBehaviourController.choose_room`: picks the room from the awareness vector (anomaly room on low evidence, memory archive on low memory confidence).
`avatar/locomotion.py`	17	Room coordinates and `target_for_room`.
`avatar/persona_mapper.py`	40	`PersonaToAvatarMapper`: maps persona and awareness to avatar tint and expression.
`persistence/journal.py`	25	World journal path and append helper (`journal_path`, `append_journal_entry`).
`persistence/provenance.py`	22	World provenance ledger (`ledger_path`, provenance ids).
`persistence/snapshots.py`	35	World snapshots (`snapshot_dir`, `create_snapshot`).

Module	Lines	Role
persistence/storage.py	267	World state directory resolution and load and save of world state.

gui/

Module	Lines	Role
branding.py	143	Shared ELI branding assets (window icon, Freedesktop theme, Windows .ico).
code_editor.py	220	Native code editor for the IDE — PySide6/PyQt, no QScintilla required.
coding_tab.py	178	ELI v2.0 — Coding tab.
panels/_qt.py	33	Shared Qt import shim for eli.gui.panels modules.
panels/permission_dialog.py	232	The consent dialog — the moment the operator actually decides.
tabs/eli_world_tab.py	11	The Eli's World tab: wraps EliWorldPanel.
tabs/experimental_tab.py	170	Read-only workbench over the experimental/ folder: lists prototype kits and opens their folders or READMEs.
tabs/marketplace_tab.py	991	Settings ▸ Plugins ▸ Marketplace — browse, scan, install, and revoke.
tabs/tasks_tab.py	284	ELI v2.0 — Tasks tab.
tabs/training_tab.py	841	ELI v2.0 — Labs ▸ Training. The GUI half of the LoRA pipeline.
widgets/eli_face.py	230	ELI's animated face — a procedural 2D face that shows the tone_adaptor's emotion.
widgets/ollama_model_selector.py	444	Ollama model selector widget for ELI GUI.
widgets/voice_downloader.py	282	Voice library browser — acquire additional Piper voices, accents and tones.

plugins/

Module	Lines	Role
calendar/plugin.py	27	Calendar plugin: hands ADD_EVENT and LIST_EVENTS to runtime/agenda.py.

Module	Lines	Role
document_reader/plugin.py	355	Document reader plugin: reads and indexes PDF and docx files.
integrity.py	291	Integrity and publisher identity for community plugins.
manifest.py	302	Plugin manifests: what a plugin says it is, checked against what it does.
marketplace.py	793	ELI's plugin marketplace client — federated, community-hosted, consent-gated.
mcp.py	481	MCP servers: one config ELI owns, and an install that proves it works.
media/plugin.py	93	Media control plugin for ELI.
notes/plugin.py	87	Notes plugin: markdown notes with NEW_NOTE, WRITE_NOTE, LIST_NOTES, SEARCH_NOTES.
permissions.py	395	Android-style consent for plugins: allow always, allow once, reject.
pomodoro/plugin.py	92	Pomodoro plugin: POMODORO_START, POMODORO_STOP, POMODORO_STATUS focus timers.
security_scan.py	547	Multi-engine malware scanner for community plugins.
subprocess_sandbox.py	308	Containment for child processes — the gap netguard structurally cannot cover.
system_stats/plugin.py	74	System stats plugin: CPU_USAGE, RAM_USAGE, SYSTEM_STATS.
tts/plugin.py	43	TTS (Text-to-Speech) plugin for ELI.
weather/plugin.py	187	Weather plugin: GET_WEATHER (geocode plus open-meteo, toggle-gated). Current conditions, or the forecast for a named day; a day word is never the place.
web/plugin.py	617	Web plugin: WEB_SEARCH (toggle-gated).
web_automation/plugin.py	92	Web Automation plugin for ELI – lazy Playwright import, safe under broken installs.

tools/

Module	Lines	Role
api.py	73	eli.api — a small, curated callable surface over ELI's action executor.
image_engine/fetch_model.py	59	Fetch a diffusers image model's weights into the local models dir.
image_engine/gui_bridge.py	453	Bridge between the GUI image tab and the engine: ImageGenerationRequest, model and preset discovery, recent outputs.
image_engine/image_engine/__main__.py	3	python -m entry for the image engine CLI.
image_engine/image_engine/cli.py	312	Command-line interface for the image engine.
image_engine/image_engine/contracts.py	126	EngineConfig, JobRequest, ArtifactRecord, JobResult.
image_engine/image_engine/plotting.py	324	Loads records and plots them with matplotlib.
image_engine/image_engine/project_analyzer.py	41	analyze_project_profile, infer_style_hints: derives a colour and style profile from a project path.
image_engine/image_engine/prompt_compiler.py	118	VisualBrief and PromptCompiler: turns a request into a structured visual brief.
image_engine/image_engine/quality.py	94	Image hashing and scoring (score_image, choose_best).
image_engine/image_engine/service.py	369	ImageEngine: runs image jobs and records artifacts.
image_engine/image_engine/visual_core.py	1446	The procedural renderer: scene types, composition, palettes, atmosphere and post-processing.
image_engine/runtime_paths.py	130	Resolves image-engine paths from the project root or the data directory.
mic_diag.py	149	ELI Microphone Diagnostic
news/news_fetcher.py	577	ELI Web Learning Module — News & Current Events Fetcher
news/news_synthesis.py	913	News synthesis cadence
registry/capabilities_doc.py	374	capabilities_doc.py — regenerate blueprints/capabilities_and_actions.md.
registry/capability_registry.py	38	Process-local capability registry (register_capability).
registry/capability_updater.py	142	capability_updater.py

package root

Module	Lines	Role
`__main__.py`	93	Console entry point. `python -m eli` and the `eli` script both call this.

`scripts/`

Module	Lines	Role
`rebuild_vector_index.py`	141	rebuild_vector_index.py

Part 5 — engine request lifecycle (`engine.process`)

The entry flow, in order:

1. **Pipeline trace id** assigned (`ELI_PIPELINE_TRACE` for observability).
2. **Prompt-injection guard FIRST** — `_eli_sanitize_user_input` runs before the router or LLM ever sees the raw text, so injected instructions can't reach them unsanitised. (A security control that sits at the very top of the pipe.)
3. **Multi-question splitter** (`kernel/engine.py`) — if a message has more than one `?`, more than 25 words and is not conversational, it splits into standalone sub-questions (each at least 6 words, at most 4) and answers each, so genuinely multi-part asks get multiple grounded answers.
4. Router → **gradient orchestrator** (all CHAT modes, depth by reasoning mode) → shared retrieval → specialist bus composition → governed output (Parts 1–4). Fallback: direct `AgentBus.dispatch()` if orchestration fails.

Note: `engine.py` still carries 34 `PHASE…` markers: layered patches whose residue (engine 16.0k / executor 15.8k / GUI 13.1k lines) is documented here at behavioural level, not line by line.

Part 6 — reasoning modes & autonomous deepening

The 5 reasoning modes were renamed (public layer; internal keys unchanged) and made adaptive + self-deepening.

Public mode	Internal key	Multi-pass algorithm	Agent time budget	Deepen iters	Confidence target
Quick	quick	single pass	1.0×	0 (never)	0.45
Normal	chain_of_thought	one reasoned pass	1.0×	1	0.55
Advanced	self_consistency	N samples + consensus	1.5×	2	0.65
Research	tree_of_thoughts	branch + prune	2.0×	3	0.75
Expert	constitutional_ai	draft → critique → revise	2.5×	4	0.80

Auto-selection (`cognition/engagement_tracker.py`): the mode is escalated automatically as a conversation deepens (Quick→Normal→Advanced→Research) without the user asking. (Internal strategy keys are stable; users only ever see the public names.)

Per-mode agent time budgets (`reasoning_modes.mode_budget_multiplier`, Stage 1b): each mode scales every agent's timeout — quick/normal unchanged, deeper modes get proportionally more time. Threaded via `intent["_mode_budget_mult"]` → `agent_bus._collect_layer`. Tunable in the Cognition tab (`cog.mode_budget_*`).

Confidence-driven iterative deepening (`runtime/grounding_escalation.escalate`, Stage 2): on a poorly-grounded *checkable-factual* turn, ELI re-dispatches the agent bus broadly, **escalating the reasoning mode one tier per iteration** and (Stage 3a) **raising the gather budget** each pass (1.5×→2.0×→2.5×), until grounding crosses the per-mode target or the per-mode iteration budget is spent (early no-improvement break). Banter/opinion/command/meta never deepen.

Background deepening (`runtime/background_deepening.py`, Stage 3b): a Quick answer returns instantly; if it was a poorly-grounded factual turn, a background thread keeps gathering (broad re-dispatch at Advanced + 2× gather) and surfaces a better answer in the Proactive panel **only if** it crosses grounding 0.55 and isn't degenerate. Tightly gated (quick-only, grounding<0.40, factual-only, deduped, 1 in-flight, 10-min cooldown). Toggle: `cog.background_deepen` / `ELI_BACKGROUND_DEEPEN`.